Hertie School

Deep Learning

Session 3: Deep neural networks (2)

September 17, 2024

Syllabus

Hertie School

Deep learning theory

1. Deep learning in public policy
2. Deep neural networks (1)
3. **Deep neural networks (2)**

Applications

4. Computer vision (1): Convolutional neural networks
5. Computer vision (2): Modern CNNs, implementation, and applications
6. Sequence methods and time-series analysis
7. Natural language processing (1)
8. Natural language processing (2): Transformer models, implementation, and applications

- Midterm exam -

9. Further topics in deep learning

Deep learning and public policy

10. Policy approaches to regulate deep learning and responsible implementation in government
11. Deep learning in practice
12. Tutorial presentations

Learning objectives

Hertie School

Deep neural networks (2)

- Backpropagation (multi-class classification, several layers)
- Mini batch gradient descent
- Training process
- Vanishing gradient problem

Hertie School

Backpropagation (continued)

Single-output neural network

Reminder

The single-layer, single-output network:

$$f(X) = o\left(b^{[2]} + \sum_{i}^{H} w_i^{[2]} \sigma\left(b_i^{[1]} + \sum_{j}^{d} w_{ij}^{[1]} x_j\right)\right)$$

Partial derivative:

$$\frac{\partial L(f(X), y)}{\partial w_{ij}^{[1]}} = \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial a^{[2]}} \cdot \frac{\partial a^{[2]}}{\partial h_i^{[1]}} \cdot \frac{\partial h_i^{[1]}}{\partial a_i^{[1]}} \cdot \frac{\partial a_i^{[1]}}{\partial w_{ij}^{[1]}}$$

Gradient update:

We plug in all the values for the rth iteration.

We update the weight using the gradient and the learning rate η

$$w_{ij}^{(r+1)} = w_{ij}^{(r)} + \eta\, \Delta w_{ij}^{(r)} = w_{ij}^{(r)} - \eta \left(\frac{\partial L(f(X), y)}{\partial w_{ij}}\right)^{(r)}$$

Computing the gradient

Multivariate chain rule

- Chains of multivariate functions are common in neural networks
- Chain rule is key ingredient for backpropagation
- Graphical representation: Nodes=variables, edges=functional dependence
- $f(u, v)$ but also $f(a, b)$ and $f(w, x, y, z)$
- **Multivariate chain rule**:

$$\frac{\partial}{\partial a} f(u(a,b), v(a,b)) = \frac{\partial f}{\partial u}\frac{\partial u}{\partial a} + \frac{\partial f}{\partial v}\frac{\partial v}{\partial a}$$

- How many terms to we need to add up to compute $\frac{\partial f}{\partial z}$?

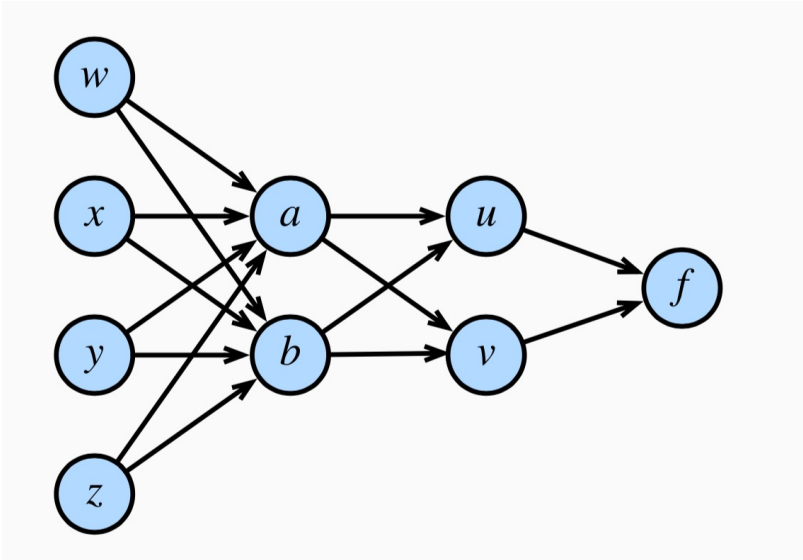


Image source: Zhang et al.

Computing the gradient

Multivariate chain rule

- Chains of multivariate functions are common in neural networks
- Chain rule is key ingredient for backpropagation
- Graphical representation: Nodes=variables, edges=functional dependence
- $f(u, v)$ but also $f(a, b)$ and $f(w, x, y, z)$
- **Multivariate chain rule**:

$$\frac{\partial}{\partial a} f(u(a,b), v(a,b)) = \frac{\partial f}{\partial u}\frac{\partial u}{\partial a} + \frac{\partial f}{\partial v}\frac{\partial v}{\partial a}$$

- How many terms to we need to add up to compute $\frac{\partial f}{\partial z}$?

Solution: $\frac{\partial f}{\partial z} = \frac{\partial f}{\partial u}\frac{\partial u}{\partial a}\frac{\partial a}{\partial z} + \frac{\partial f}{\partial v}\frac{\partial v}{\partial a}\frac{\partial a}{\partial z} + \frac{\partial f}{\partial u}\frac{\partial u}{\partial b}\frac{\partial b}{\partial z} + \frac{\partial f}{\partial v}\frac{\partial v}{\partial b}\frac{\partial b}{\partial z}$

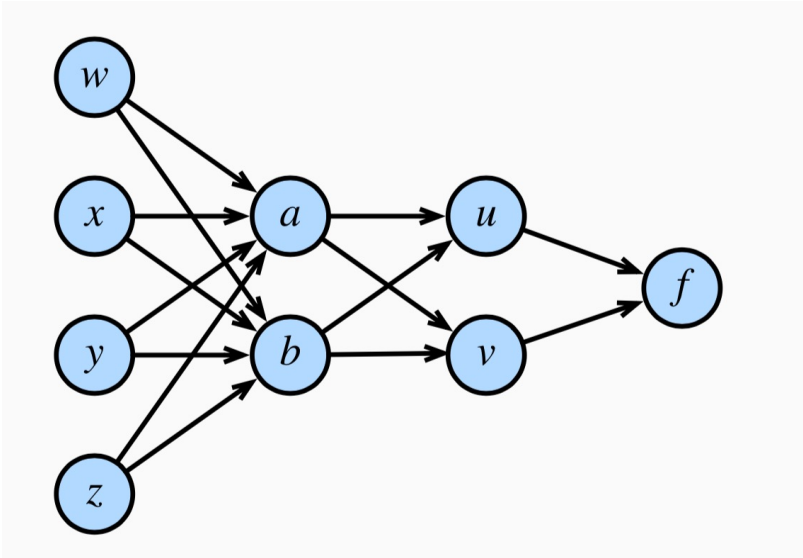


Image source: Zhang et al.

Multiple output nodes

Multi-class classification

Single-layer neural network:

$$f_k(X) = o\left(b_k^{[2]} + \sum_i^H w_{ki}^{[2]} \sigma\left(b_i^{[1]} + \sum_j^d w_{ij}^{[1]} x_j\right)\right)_k$$

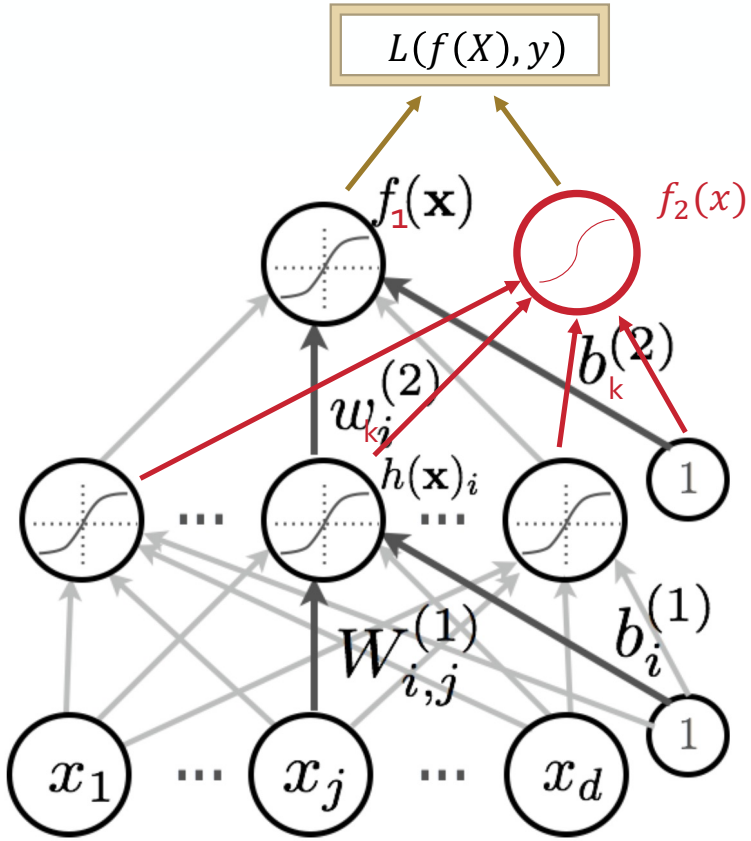
Cross-entropy loss:

$$L(f(X), y) = -\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik} \log f_k(x_i)$$

Gradient?

$$\nabla L(f(X), y)$$

$$f(X) = (f_1(X), f_2(X))$$



Example with 2 output nodes (K=2)

Multiple output nodes

Multivariate chain rule:

$$\frac{\partial}{\partial a} f(u(a,b), v(a,b))$$

$$= \frac{\partial f}{\partial u}\frac{\partial u}{\partial a} + \frac{\partial f}{\partial v}\frac{\partial v}{\partial a}$$

Multi-class classification

Applying the multivariate chain rule:

$$\frac{\partial L(f(X), y)}{\partial w_{ij}} = \sum_{k=1}^{K} \frac{\partial L}{\partial f_k} \cdot \frac{\partial f_k}{\partial a_k^{[2]}} \cdot \frac{\partial a_k^{[2]}}{\partial h_i^{[1]}} \cdot \frac{\partial h_i^{[1]}}{\partial a^{[1]}} \cdot \frac{\partial a^{[1]}}{\partial w_{ij}^{[1]}}$$

Example cross-entropy loss:

$$\frac{\partial L(f(X), y)}{\partial f_k} = -\frac{\partial}{\partial f_k} \sum_{i=1}^{N} \sum_{k=1}^{K} y_{ik} \log f_k(x_i)$$

$$= -\sum_{i=1}^{N} y_{ik} \frac{1}{f_k(x_i)}$$

Trick: Often the cross-entropy loss is combined with the softmax function and we directly compute $\frac{\partial L}{\partial a_k}$!

Hertie School

Multiple output nodes

Cross-entropy loss and softmax

Cross-entropy loss and softmax activation are often combined for classification.
Here a_k is the output activation for the final layer and class k:

$$L(y,\hat{y}) = -\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik} \ln\ f_k(x_i) = -\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik} \ln \frac{e^{a_k}}{\sum_{l=1}^{K} e^{a_l}}$$

$$= \sum_{i=1}^{N}\left(\sum_{k=1}^{K} y_{ik} \ln \sum_{l=1}^{K} e^{a_l} - \sum_{k=1}^{K} y_{ik} a_k\right)$$

$$= \sum_{i=1}^{N}\left(\left(\ln \sum_{l=1}^{K} e^{a_l}\right)\sum_{k=1}^{K} y_{ik} - \sum_{k=1}^{K} y_{ik} a_k\right) \qquad \text{with } \sum_{k=1}^{K} y_{ik} = 1$$

$$= \sum_{i=1}^{N}\left(\ln \sum_{l=1}^{K} e^{a_l} - \sum_{k=1}^{K} y_{ik} a_k\right)$$

Multiple output nodes

Cross-entropy loss and softmax

The derivative with respect to the output layer activation simplifies a lot:

$$\frac{\partial L}{\partial a_k} = \frac{\partial}{\partial a_k}\left[\sum_{i=1}^{N}\left(\ln\sum_{l=1}^{K} e^{a_l} - \sum_{k=1}^{K} y_{ik}a_k\right)\right]$$

$$= \sum_{i=1}^{N}\left(\frac{\partial}{\partial a_k}\ln\sum_{l=1}^{K} e^{a_l} - \frac{\partial}{\partial a_k}\sum_{k=1}^{K} y_{ik}a_k\right)$$

$$= \sum_{i=1}^{N}\left(o(a)_k - \frac{\partial}{\partial a_k}\sum_{k=1}^{K} y_{ik}a_k\right)$$

$$= \sum_{i=1}^{N}(o(a)_k - y_{ik})$$

$$= \sum_{i=1}^{N}(f_k - y_{ik})$$

with

$$\frac{\partial}{\partial a_k}\ln\sum_{l=1}^{K} e^{a_l}$$

$$= \frac{1}{\sum_{l=1}^{K} e^{a_l}}\frac{\partial}{\partial a_k}\sum_{l=1}^{K} e^{a_l}$$

$$= \frac{1}{\sum_{l=1}^{K} e^{a_l}} e^{a_k} = o(a)_k$$

Multi-layer neural network

Two hidden-layer neural network (Multilayer Perceptron)

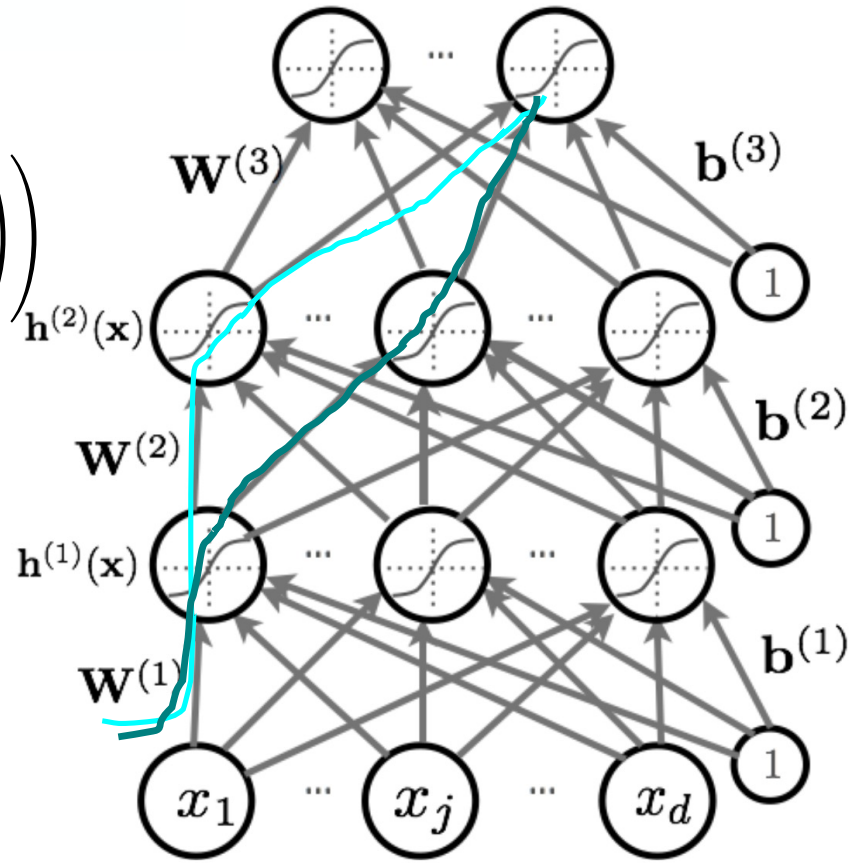
The network:

$$f_k(\boldsymbol{x}) = o\left(b_k^{[3]} + \sum_{l=1}^{H^{[2]}} w_{kl}^{[3]} g\left(b_l^{[2]} + \sum_{i=1}^{H^{[1]}} w_{li}^{[2]} g\left(b_i^{[1]} + \sum_{j}^{d} w_{ij}^{[1]} x_j\right)\right)\right)$$

Using the multivariate chain rule:

$$\frac{\partial L(f(\boldsymbol{x}), y)}{\partial w_{ij}^{[1]}} = \sum_{k=1}^{K} \frac{\partial L}{\partial f_k} \cdot \frac{\partial f_k}{\partial w_{ij}^{[1]}}$$

$$= \sum_{k=1}^{K} \frac{\partial L}{\partial f_k} \cdot \frac{\partial f_k}{\partial a_k^{[3]}} \cdot \sum_{l=1}^{H^{[2]}} \frac{\partial a_k^{[3]}}{\partial h_l^{[2]}} \frac{\partial h_l^{[2]}}{\partial a_l^{[2]}} \cdot \frac{\partial a_l^{[2]}}{\partial h_i^{[1]}} \frac{\partial h_i^{[1]}}{\partial a_i^{[1]}} \cdot \frac{\partial a_i^{[1]}}{\partial w_{ij}^{[1]}}$$



Hertie School

Vectorization

Vectorization

Scalar multiplications

$$\sum_{j=1}^{d} w_j \cdot x_j$$

For the computer:

```
sum = 0
for j in range(d):
        result = w[j]*x[j]
        sum = sum + result
```

Vector multiplications

Scalar product of two vectors of length d

$$\sum_{j=1}^{d} w_j \cdot x_j = \boldsymbol{w} \cdot \boldsymbol{x}$$

For the computer:

```
sum=np.dot(w, x)
```

- No repetitions of instructions
- Can more efficiently execute multiple operations in parallel
- Faster and more efficient use of hardware

Vectorizing the neural network

Single-layer neural network:

$$f_k(X) = o\left(b_k^{[2]} + \sum_i^H w_{ki}^{[2]} g\left(b_i^{[1]} + \sum_j^d w_{ij}^{[1]} x_j\right)\right)_k$$

Vectorized:

$$\underset{\boldsymbol{K}}{\boldsymbol{f}(x)} = o\left(\underset{\boldsymbol{K}}{\boldsymbol{b}^{[2]}} + \underset{\boldsymbol{K}\times\boldsymbol{H}}{W^{[2]}}\underset{\boldsymbol{H}}{\boldsymbol{h}^{[1]}}\right)$$

$$\underset{\boldsymbol{H}}{\boldsymbol{h}^{[1]}} = g\left(\underset{\boldsymbol{H}}{\boldsymbol{b}^{[1]}} + \underset{\boldsymbol{H}\times\boldsymbol{d}}{W^{[1]}}\underset{\boldsymbol{d}}{\boldsymbol{x}}\right)$$

Vectorizing the neural network

Single-layer neural network:

$$f_k(X) = o\left(b_k^{[2]} + \sum_i^H w_{ki}^{[2]} g\left(b_i^{[1]} + \sum_j^d w_{ij}^{[1]} x_j\right)\right)_k$$

Writing this even more compact, we include the bias in the weight matrix:

$$\boldsymbol{f}(x) = o\left(W^{[2]} \quad g\left(W^{[1]} \quad \boldsymbol{x}\right)\right)$$

Also, the input data now has one more component $\boldsymbol{x} \in \mathbb{R}^{d+1}$

$$\boldsymbol{x} = (1, x_1, \dots, x_d)$$

Vectorizing the neural network

Single-layer neural network:

$$f_k(X) = o\left(b_k^{[2]} + \sum_i^H w_{ki}^{[2]} g\left(b_i^{[1]} + \sum_j^d w_{ij}^{[1]} x_j\right)\right)_k$$

Writing this even more compact, we include the bias in the weight matrix:

$$\underset{\boldsymbol{K}}{\boldsymbol{f}(x)} = o\left(\underset{\boldsymbol{K}\times(\boldsymbol{H}+\boldsymbol{1})}{W^{[2]}} \; g\left(\underset{(\boldsymbol{H}+\boldsymbol{1})\times(\boldsymbol{d}+\boldsymbol{1})}{W^{[1]}} \; \underset{(\boldsymbol{d}+\boldsymbol{1})}{\boldsymbol{x}}\right)\right)$$

Also, the input data now has one more component $\boldsymbol{x} \in \mathbb{R}^{d+1}$

$$\boldsymbol{x} = (1, x_1, \dots, x_d)$$

$$W^{[1]} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ b_1 & w_{11}^{[1]} & \dots & w_{1d}^{[1]} \\ \dots & \dots & w_{ij}^{[1]} & \dots \\ b_H & w_{Hj}^{[1]} & \dots & w_{Hd}^{[1]} \end{bmatrix}$$

Vectorizing the neural network

Hertie School

Note in the problem set and across literature:
$a^{[l]}$ is also called $Z^{[l]}$
$h^{[l]}$ is also called $a^{[l]}$

Forward pass: 2-hidden-layer neural network

Two-hidden-layer neural network:

$$\boldsymbol{f}(x) = o\left(W^{[3]}\, g\left(W^{[2]}\, g\left(W^{[1]}\, \boldsymbol{x}\right)\right)\right)$$

Forward pass: L-hidden-layer neural network

More generally, we can write the L-layer neural network/MLP as

$$\boldsymbol{f}(x) = o\left(W^{[L]}\, g\left(W^{[L-1]}\, \ldots g\left(W^{[1]}\, \boldsymbol{x}\right)\right)\right)$$

$$= o\left(W^{[L]}\, g\left(W^{[L-1]}\, \ldots g\left(\boldsymbol{a}^{[1]}\right)\right)\right)$$

$$= o\left(W^{[L]}\, g\left(W^{[L-1]}\, \boldsymbol{h}^{[L-2]}\right)\right) = o(\boldsymbol{a}^{[L]})$$

Hertie School

Backpropagation vectorized

Backpropagation gets a lot more involved to derive in vectorized writing using more advanced linear algebra. However, the solution can be written compactly with the error signal.

Last layer:

$$\nabla_{a^{[L]}}\mathcal{L} = \frac{\partial L}{\partial \boldsymbol{f}} \cdot o'\left(a^{[L]}\right) \equiv \delta^{[L]}$$

$$\nabla_{W^{[L]}}\mathcal{L} = \frac{\partial L}{\partial \boldsymbol{f}} \cdot o'\left(a^{[L]}\right)\frac{da^{[L]}}{dW^{[L]}} = \delta^{[L]}h^{[L-1]}$$

For the other layers, we work recursively:

$$\delta^{[l]} = \left(W^{[l+1]}\right)^T \cdot \delta^{[l+1]} \cdot g'\left(a^{[l]}\right)$$

$$\nabla_{W^{[l]}}\mathcal{L} = \delta^{[l]}h^{[l-1]}$$

Dimensions of gradient:

$$\left(\boldsymbol{H^l}+\mathbf{1}\right)\times\left(\boldsymbol{H^{l-1}}+\mathbf{1}\right)$$

$$\left(\boldsymbol{H^l}+\mathbf{1}\right)\times\mathbf{1}$$

$$\mathbf{1}\times\left(\boldsymbol{H^{l-1}}+\mathbf{1}\right)$$

Refer to for example Stanford CS229 notes for details on derivatives https://cs229.stanford.edu/main_notes.pdf

Hertie School

Minibatch stochastic gradient descent

Parallelization

Stochastic gradient descent

For every weight update r:
Compute the gradient for one datapoint i and update

$$W^{(r+1)} = W^{(r)} + \eta^{(r)}\, \Delta W^{(r)}$$

- For-loop with one computation at a time sequentially

Batch gradient descent

For every weight update r:
Compute the gradient for all datapoints $i, \dots, n$ and update

$$W^{(r+1)} = W^{(r)} + \eta^{(r)} \frac{1}{n} \sum_{i=1}^{n} \left(\Delta W^{(r)}\right)_i$$

- The weight updates $\left(\Delta w^{(r)}\right)_i$ in each epoch can be computed in parallel
- Much more training data can be processed per unit of time

Vectorizing the neural network

Batch gradient descent

In batch gradient descent, we process the whole training dataset before we update the weights.

$$W^{(r+1)} = W^{(r)} + \eta^{(r)} \frac{1}{n} \sum_{i=1}^{n} \left(\Delta W^{(r)} \right)_i$$

Forward pass over all datapoints:

$$f(x) = o\left(\underset{\boldsymbol{K \times (H+1)}}{W^{[2]}} \underbrace{\underset{\boldsymbol{(H+1)\times(d+1)}}{g\left(W^{[1]} \right.} \underset{\boldsymbol{(d+1)\times n}}{\left. X \right)}}_{\boldsymbol{(H+1)\times n}} \right)$$

$\boldsymbol{K \times n}$

- For large datasets, vectorizing computations may result in memory issues
- Slower to converge than SGD since we update only after a pass through the full training set

Mini-batches

Introducing mini-batches

Our training dataset has the form of $X \in \mathbb{R}^{d \times n}$ and $Y \in \mathbb{R}^{1 \times n}$

Mini-batches are smaller subsets of the dataset of size B.

We divide the whole training data into m of these mini-batches of equal size:

$$X = [X^{\{1\}}, \dots, X^{\{m\}}]$$

Minibatch t: $(X^{\{t\}}, Y^{\{t\}})$ with $X^{\{t\}} \in \mathbb{R}^{d \times B}$ and $Y^{\{t\}} \in \mathbb{R}^{1 \times B}$

Q: If we have a dataset of 600,000 datapoints and we use a minibatch size of 100, how many mini batches do we have?

Mini-batches

Introducing mini-batches

Our training dataset has the form of $X \in \mathbb{R}^{d \times n}$ and $Y \in \mathbb{R}^{1 \times n}$

Mini-batches are smaller subsets of the dataset of size B.

We divide the whole training data into m of these mini-batches of equal size:

$$X = [X^{\{1\}}, \dots, X^{\{m\}}]$$

Minibatch t: $(X^{\{t\}}, Y^{\{t\}})$ with $X^{\{t\}} \in \mathbb{R}^{d \times B}$ and $Y^{\{t\}} \in \mathbb{R}^{1 \times B}$

Q: If we have a dataset of 600,000 datapoints and we use a minibatch size of 100, how many mini batches do we have? A: 6000

Mini-batch gradient descent

Typical mini-batch size is $B = 32, 64, 128, \ldots$ Powers of 2 can be more easily processed by CPU/GPU

Algorithm

Process like batch gradient descent, just on one mini-batch at a time.
We have m mini-batches of size B.

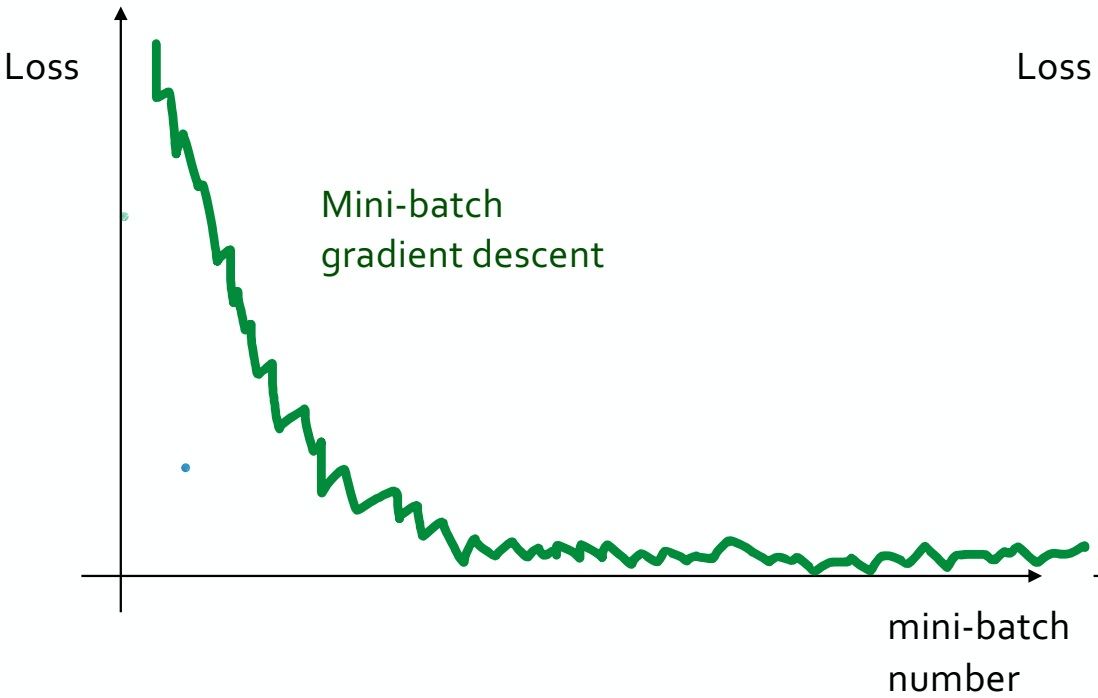
For each epoch:

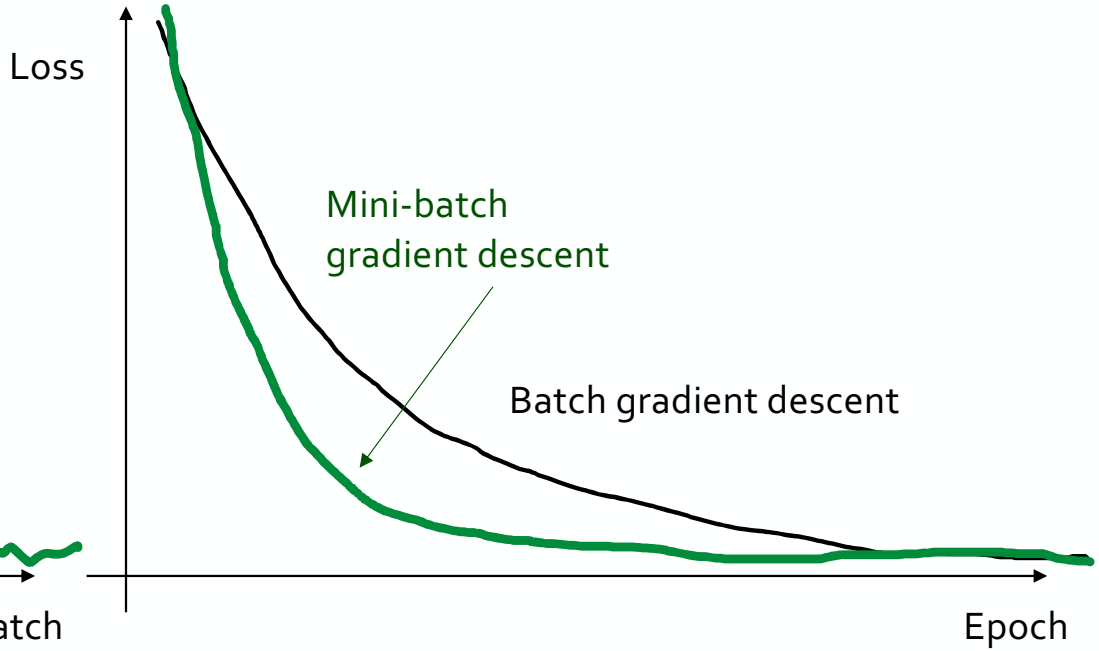
For every $t = 1, \ldots, m$, perform mini-batch gradient descent by

1. doing a **forward-pass** to get the predictions based on the current values of the parameters
2. computing the loss
3. computing the gradient for current values of the weights and mini-batches in a **backward pass**
4. updating the parameters $W^{(r+1)} = W^{(r)} + \eta^{(r)} \frac{1}{B} \sum_{i=1}^{B} \left(\Delta W^{(r)} \right)_i$

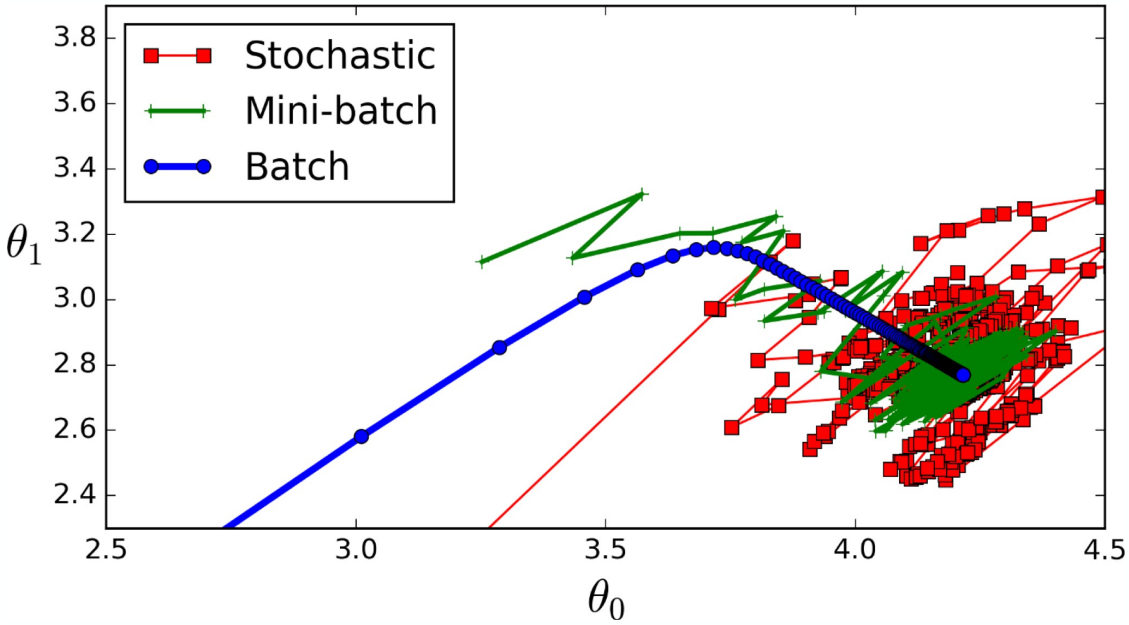
Mini-batch gradient descent

Training error curve





Convergence of gradient descent



Different optimization approaches

Batch GD:

- Process whole training set in each update step
- Low variance
- For large datasets or high-dimensional data becomes very slow

Stochastic GD (SGD):

- One data point at each step
- High variance; oscillation around the optimum provides regularization
- Choose small learning rates

Mini-batch GD:

- Combines both approaches above
- Exploits computational efficiency

Hertie School

Training process

Generalization

Supervised learning problem formulation

- Most often assume that data are independent and identically distributed (IID) random variables from distribution $P(X, Y)$.
- We want a model learned on this data to work well on out-of-sample data from the same distribution - to generalize
- **Training error** is a function of the training data and the model

$$R_{train}[X, y, f] = \frac{1}{n}\sum_{i=1}^{n} L(x_i, y_i, f(x_i))$$

- The true **generalization error** is an expectation taken with respect to the underlying distribution

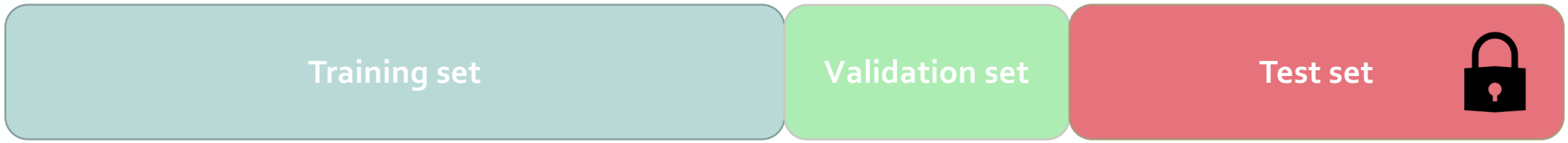
$$R[p, f] = E_{(x,y)\sim P}[L(x, y, f(x)] = \iint L\big(x, y, f(x)\big)p(x, y)dxdy$$

- Since we don't know the true distribution P, we need to **estimate** the generalization using a test set, and computing it like the training error.

Sometimes, out of sample data come from a different distribution $Q(\cdot) \neq P(\cdot)$. We will talk about distribution shifts later.

Generalization

Training, validation and testing sets



- Typical modelling setup for training deep learning models is train, validation, and test set.
- The test set remains locked away until the end of the experiments to report the final performace of the model.
- We use the validation set (or several) for hyperparameter tuning and model selection.
- Cross-validation also used for hyperparameter tuning and model selection.
- Cross-validation often sometimes as practical in deep learning since it can be computationally intensive.

Generalization

Partner exercise

Q: What does a low training error tell us about generalization?

Q: What does it mean if the training error does not decrease considerably, and training and validation error are similar?

Q: What does it mean if the training error is significantly lower than the validation error?

Generalization

Partner exercise

Q: What does a low training error tell us about generalization?
A: A low training error does not mean the model generalizes well. Test error can be also low or high. We need to look at the test error for generalization.

Q: What does it mean if the training error does not decrease considerably, and training and validation error are similar?
A: This means that the model is unable to learn the pattern from the data, e.g. because it is too simple (underfitting).

Q: What does it mean if the training error is significantly lower than the validation error?
A: We are overfitting to the training data.

Generalization

Model complexity

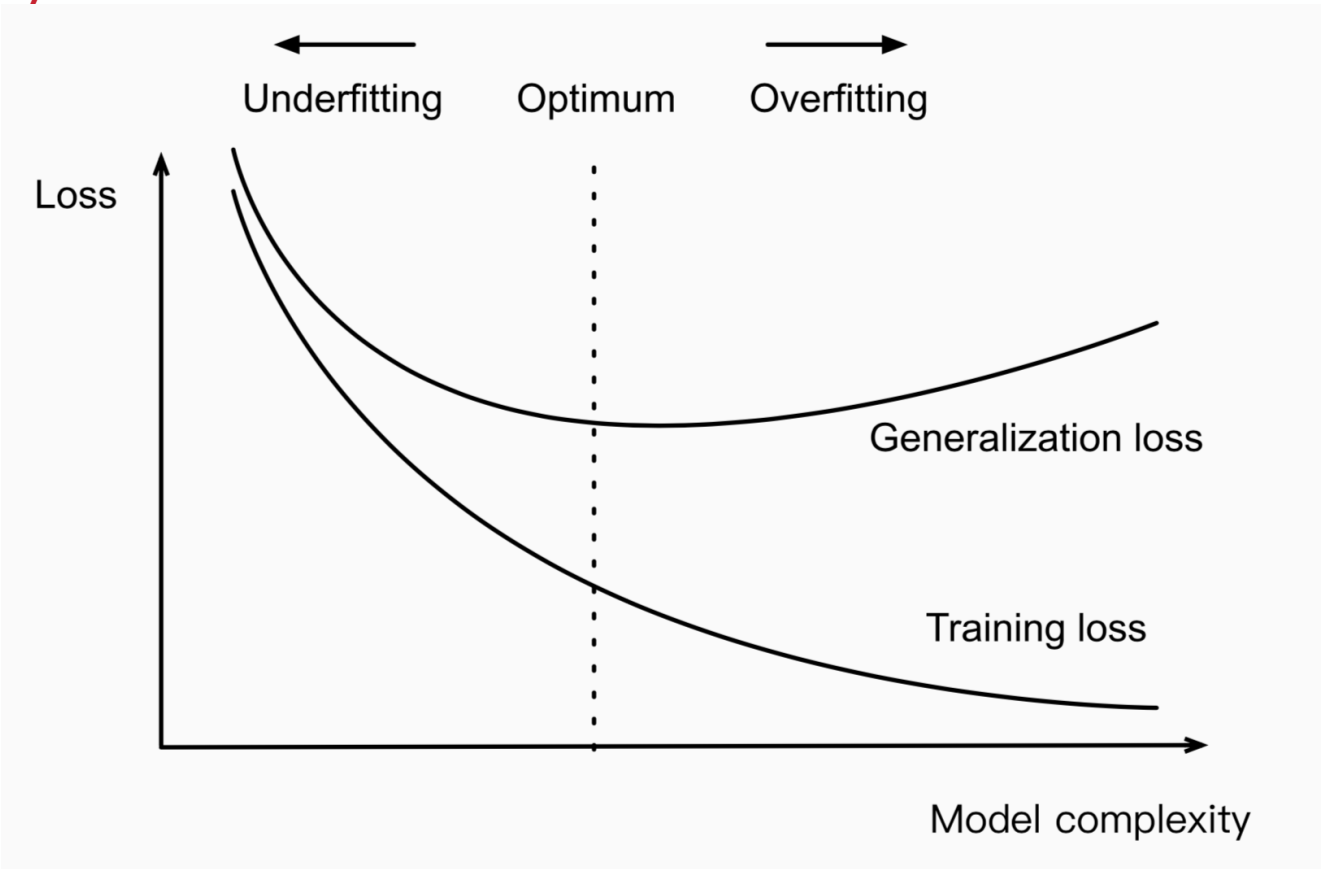
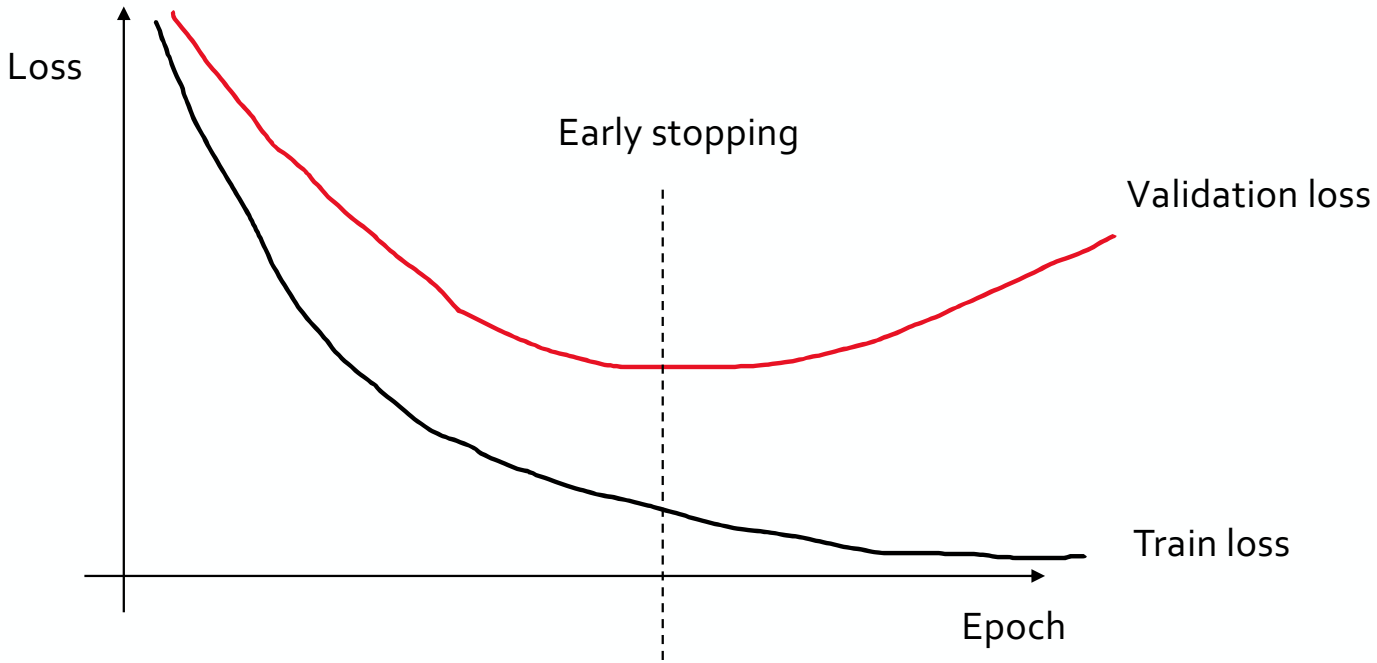


Image: Zhang et al.

Early stopping

Under- and overfitting by training time

- One epoch refers to one pass over the entire training data



Hertie School

Performance metrics common in deep learning

Binary classification metrics

$$\text{Accuracy} = \frac{\text{correct classifications}}{\text{all classifications}} = \frac{\sum_{k=1}^{K} \frac{TP_k + TN_k}{TP_k + FP_k + TN_k + FN_k}}{K}$$

Accuracy may misrepresent performance for imbalanced classes.

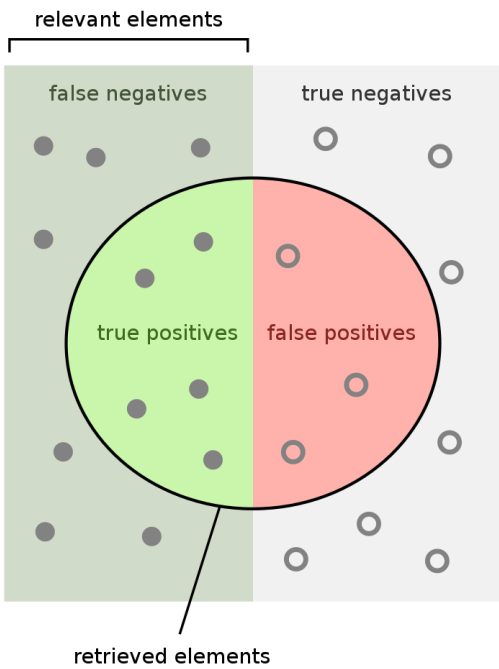
$$\text{Precision} = \frac{\text{correct positive classifications}_\text{k}}{\text{all positive classifications}_\text{k}} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{\text{correct positive classifications}}{\text{all true positives}} = \frac{TP}{TP + FN}$$

Aggregate measure to deal with imbalanced classes (weighted harmonic mean of precision and recall):

$$F_\beta - score = \frac{(\beta^2 + 1) precision \cdot recall}{\beta^2\, precision + recall} = \frac{(\beta^2 + 1)TP}{(\beta^2 + 1)TP + \beta^2 FN + FP}$$

$\beta > 1$ weighs recall higher than precision, $\beta = 1$ corresponds to F1-score



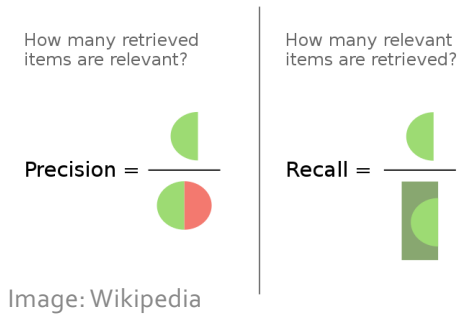


Image: Wikipedia

Performance metrics

Hertie School

There is a range of different metrics

Sources: [4][5][6][7][8][9][10][11][12] view·talk·edit

		Predicted condition			
	Total population $= P + N$	**Positive (PP)**	**Negative (PN)**	Informedness, bookmaker informedness (BM) $= TPR + TNR - 1$	Prevalence threshold (PT) $= \frac{\sqrt{TPR \times FPR} - FPR}{TPR - FPR}$
Actual condition	**Positive (P)**	**True positive (TP),** **hit**	**False negative (FN),** **type II error, miss,** **underestimation**	True positive rate (TPR), recall, sensitivity (SEN), probability of detection, hit rate, power $= \frac{TP}{P} = 1 - FNR$	False negative rate (FNR), miss rate $= \frac{FN}{P} = 1 - TPR$
	Negative (N)	**False positive (FP),** **type I error, false alarm,** **overestimation**	**True negative (TN),** **correct rejection**	False positive rate (FPR), probability of false alarm, fall-out $= \frac{FP}{N} = 1 - TNR$	True negative rate (TNR), specificity (SPC), selectivity $= \frac{TN}{N} = 1 - FPR$
	Prevalence $= \frac{P}{P + N}$	Positive predictive value (PPV), precision $= \frac{TP}{PP} = 1 - FDR$	False omission rate (FOR) $= \frac{FN}{PN} = 1 - NPV$	Positive likelihood ratio (LR+) $= \frac{TPR}{FPR}$	Negative likelihood ratio (LR−) $= \frac{FNR}{TNR}$
	Accuracy (ACC) $= \frac{TP + TN}{P + N}$	False discovery rate (FDR) $= \frac{FP}{PP} = 1 - PPV$	Negative predictive value (NPV) $= \frac{TN}{PN}$ $= 1 - FOR$	Markedness (MK), deltaP (Δp) $= PPV + NPV - 1$	Diagnostic odds ratio (DOR) $= \frac{LR+}{LR-}$
	Balanced accuracy (BA) $= \frac{TPR + TNR}{2}$	F_1 score $= \frac{2PPV \times TPR}{PPV + TPR} = \frac{2TP}{2TP + FP + FN}$	Fowlkes–Mallows index (FM) $= \sqrt{PPV \times TPR}$	Matthews correlation coefficient (MCC) $= \sqrt{TPR \times TNR \times PPV \times NPV} - \sqrt{FNR \times FPR \times FOR \times FDR}$	Threat score (TS), critical success index (CSI), Jaccard index $= \frac{TP}{TP + FN + FP}$

Image: Wikipedia

Performance metrics

ROC AUC (Area under the ROC curve)

True Positive Rate/recall

$$TPR = \frac{TP}{TP \quad + FN}$$

False Positive Rate

$$FPR = \frac{FP}{FP + TN}$$

- AUC is **scale-invariant**. It measures how well predictions are ranked, rather than their absolute values.
- AUC is **classification-threshold-invariant**. It allows to evaluate the model's performance by considering all classification thresholds simultaneously. – But sometimes we want intuitive thresholds (e.g. 0.5)

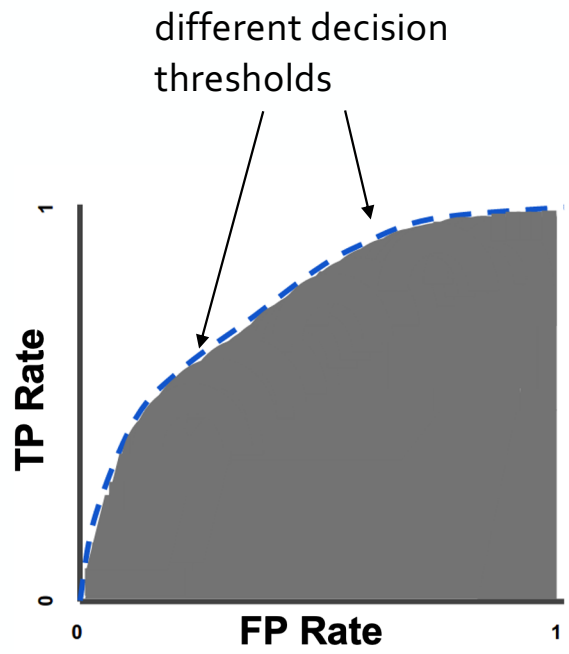


Image: Google

Performance metrics common in deep learning

(Multi-class) classification metrics

Compute the metrics per class, then…

macro-average over all classes:

$$\text{Precision}_{\text{M}} = \frac{\sum_{k=1}^{K} \frac{TP_k}{TP_k + FP_k}}{K}$$

micro-average over all classes:

$$\text{Precision}_{\mu} = \frac{\sum_{k=1}^{K} TP_k}{\sum_{k=1}^{K} TP_k + FP_k}$$

Macro-averaging treats all classes equally while micro-averaging favors bigger classes.

The average F-score can be computed using these average values for precision and recall

$$F - score_M = \frac{(\beta^2 + 1) precision_M \, recall_M}{\beta^2 \, precision_M + recall_M}$$

Training tips

Hertie School

You want your model to be able to overfit!

Underfitting

Problem: Training error doesn't decrease.

- Increase model complexity/change model type.
- Optimization doesn't work, change it with
  - Momentum in gradient descent
  - Batch normalization (each unit's pre-activation is normalized)
  - Change learning rate (increase it, or adaptive learning rate)
  - Use ReLU instead of sigmoid/tanh → vanishing gradient
  - Initialize weights to avoid saturation of activation functions across units.
- Check for bug in the code.

Training tips

Overfitting

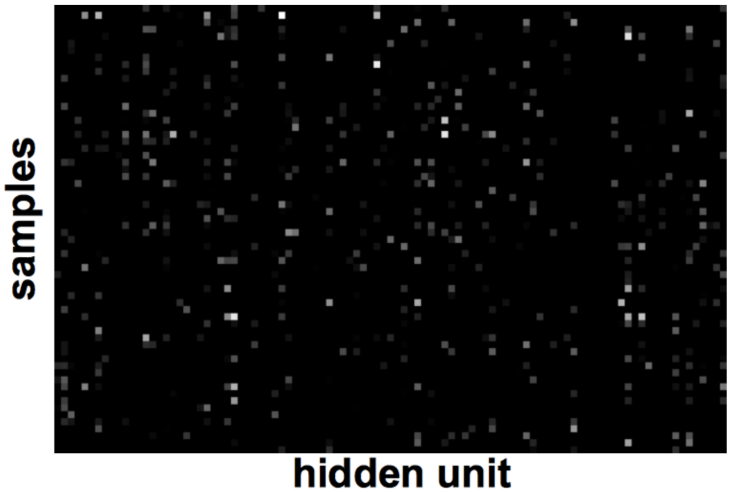
Problem: Training error very low, poor validation performance.

- Use regularization techniques to reduce the model complexity (*discussed in a later session*)
    - Weigth sharing (which greatly reduces the number of parameters)
    - Dropout
    - Weight decay
    - Sparsity in the hidden units

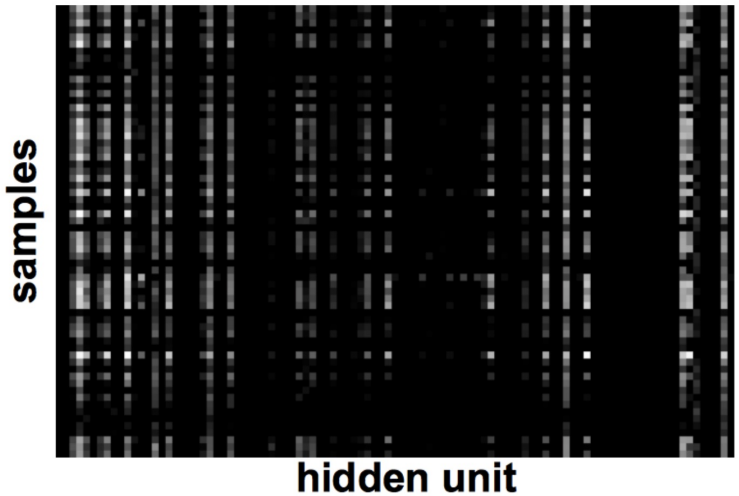
Training tips

Visualizing features

Visualize parameters: learned features should exhibit structure and should be uncorrelated



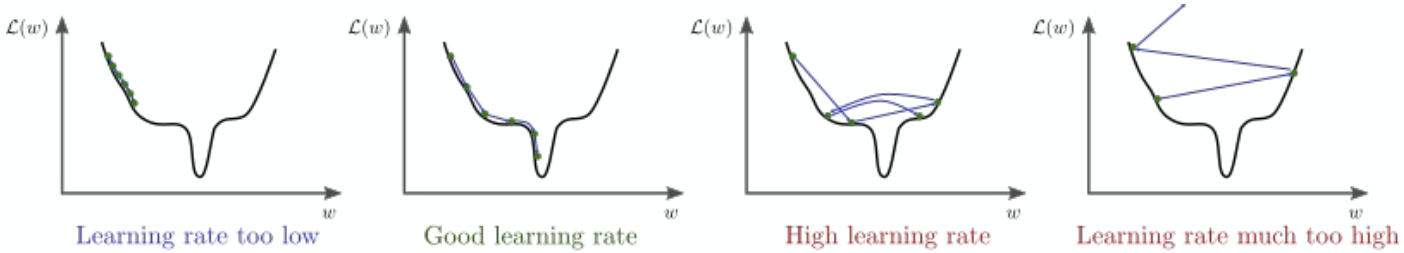
Good training: hidden units are sparse across samples



Bad training: many hidden units ignore the input and/or exhibit strong correlations

From Marc'Aurelio Ranzato, CVPR 2014 tutorial

Training tips



Other things that can go wrong

- Training error rate diverges:
  - Learning rate may be too large

    → decrease learning rate
  - Bug in the backpropagation

    → numerical gradient checking
- Loss is minimized but accuracy is low

  → check loss function: Is it appropriate for the task you want to solve?

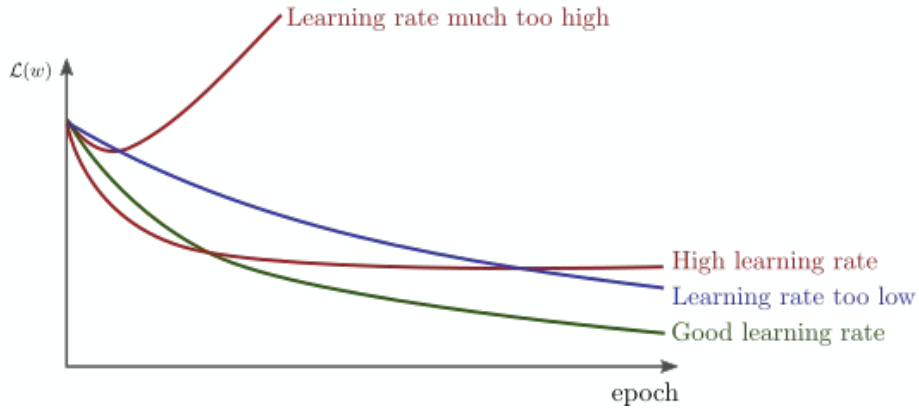


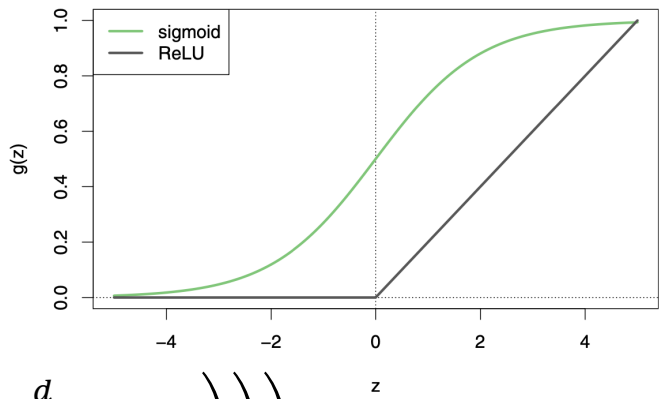
Image: Stanford cs231n

Hertie School

Vanishing gradient problem

Vanishing gradient problem

Saturation



Sigmoid activation function

Two-layer neural network

$$f_k(X) = o\left(b_k^{[3]} + \sum_{l=1}^{H^{[2]}} w_{kl}^{[3]} \sigma\left(b_l^{[2]} + \sum_{i=1}^{H^{[1]}} w_{li}^{[2]} \sigma\left(b_i^{[1]} + \sum_{j}^{d} w_{ij}^{[1]} x_j\right)\right)\right)$$

Component of the gradient:

$$\frac{\partial L(f(X), y)}{\partial w_{ij}^{[1]}} = \sum_{k=1}^{K} \frac{\partial L}{\partial f_k} \cdot \frac{\partial f_k}{\partial a_k^{[3]}} \cdot \sum_{l=1}^{H^{[2]}} \frac{\partial a_k^{[3]}}{\partial h_l^{[2]}} \frac{\partial h_l^{[2]}}{\partial a_l^{[2]}} \cdot \frac{\partial a_l^{[2]}}{\partial h_i^{[1]}} \frac{\partial h_i^{[1]}}{\partial a_i^{[1]}} \cdot \frac{\partial a_i^{[1]}}{\partial w_{ij}^{[1]}}$$

$$= \sum_{k=1}^{K} \frac{\partial L}{\partial f_k} \cdot \frac{\partial f_k}{\partial a_k^{[3]}} \cdot \sum_{l=1}^{H^{[2]}} \frac{\partial a_k^{[3]}}{\partial h_l^{[2]}} \cdot \sigma(a_l^{[2]})\left(1 - \sigma(a_l^{[2]})\right) \cdot \frac{\partial a_l^{[2]}}{\partial h_i^{[1]}} \sigma(a_i^{[1]})\left(1 - \sigma(a_i^{[1]})\right) \cdot \frac{\partial a_i^{[1]}}{\partial w_{ij}^{[1]}}$$

What happens if we multiply many of the sigmoid activation functions with each other?

Saturation

Sigmoid activation function

"Firing" neuron means $\sigma(a)$ is close to 1
Not activated neuron means $\sigma(a)$ is close to 0.

$\rightarrow$ the derivative $\sigma(a)(1-\sigma(a))$ is very small!

For every layer, we multiply another very small number to compute the gradient, resulting in an infinitesimal gradient for a large number of layers.

This is called the **vanishing gradient problem:** The gradient becomes so small that weights don't really change their values during the update.

Occurs similarly with tanh activation functions.

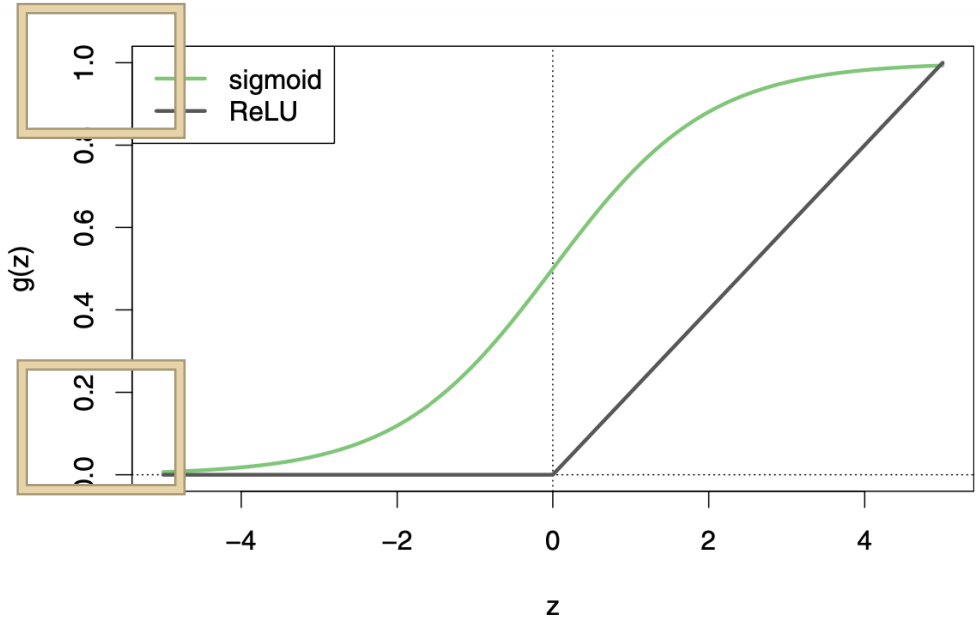


Image: ISLR

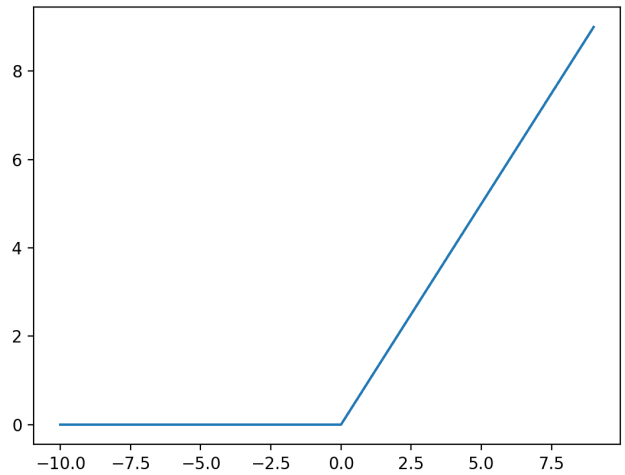
Hertie School

Overcoming the vanishing gradient problem

Other activation functions

ReLU activation function is non-saturating

$$g(z) = \begin{cases} 0 & \text{if } z < 0 \\ z & \text{otherwise} \end{cases}$$



Widely used in deep learning because of this property and the efficienty in computation.

Hertie School

Overcoming the vanishing gradient problem

Batch normalization

- Normalizing the activation to keep it in a non-saturating regime
- Each minibatch is normalized to zero mean and unit variance with the sample mean and sample standard deviation of the mini batch
- Simplified, for a fully connected layer with input X the mini-batch normalization BN plays out like this

$$h = g(BN(b + Wx))$$

- Additional scale parameter and shift parameter are introduced as part of BN, which are learned from the data as well.

See also 8.5 in Zhang et al.

Hertie School

Overcoming the vanishing gradient problem

Residual networks

Skip connections jump over activation functions

Conventional chaining

$$h^l = g\left(W^{[l]} \cdot g\left(W^{[l-1]} \cdot g\left(W^{[l-2]} \cdot h^{[l-3]}\right)\right)\right)$$

Residual networks skipping one layer

$$h^l = g\left(W^{[l]} \cdot h^{[l-1]} + W^{[l-1]} \cdot h^{[l-2]}\right)$$

Residual networks skipping two layers

$$h^l = g\left(W^{[l]} \cdot h^{[l-1]} + W^{[l-2]} \cdot h^{[l-3]}\right)$$

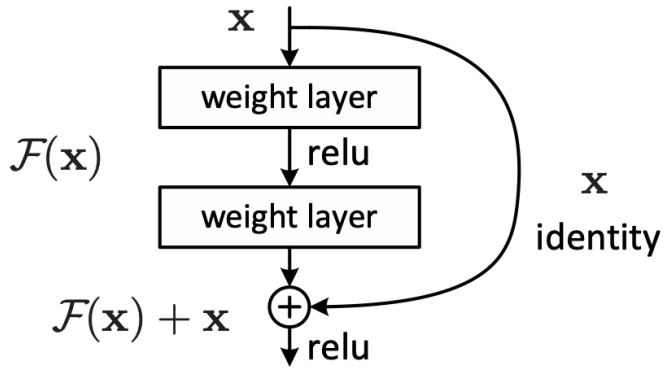


Figure 2. Residual learning: a building block.

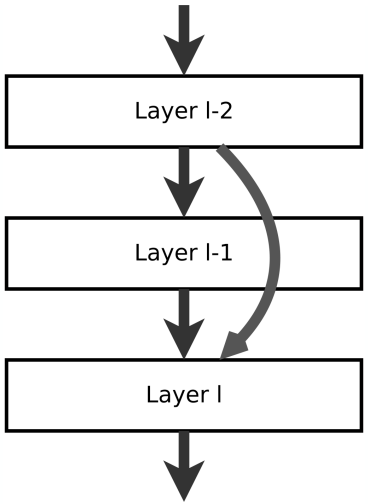


Image: He, Zhang, Ren & Sun (2015)

Vanishing gradient problem

Overcoming the vanishing gradient problem

Residual networks

Implementation in computer vision models

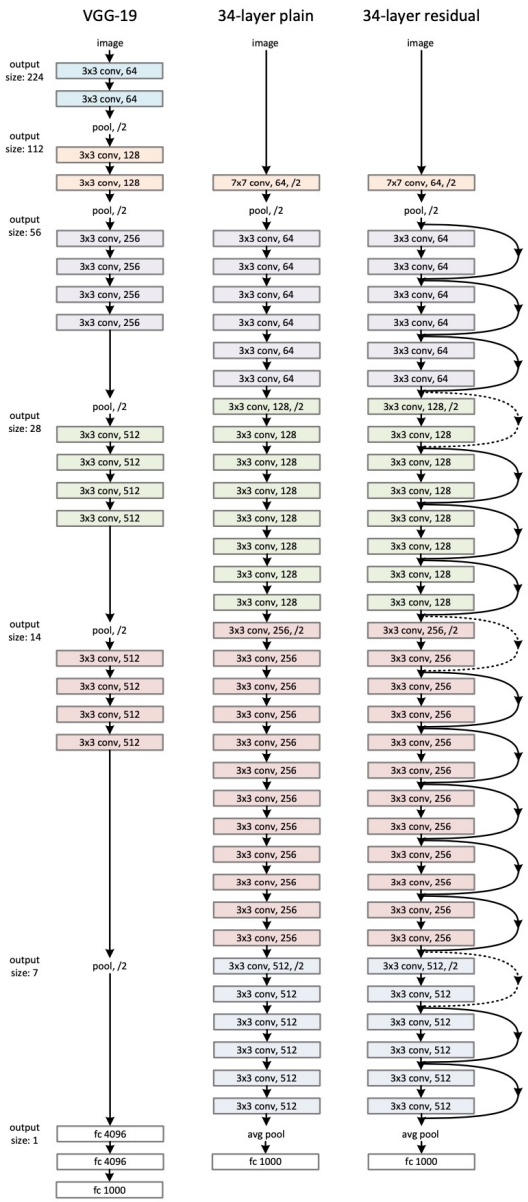


Figure 3. Example network architectures for ImageNet. Left: the VGG-19 model [41] (19.6 billion FLOPs) as a reference. Middle: a plain network with 34 parameter layers (3.6 billion FLOPs). Right: a residual network with 34 parameter layers (3.6 billion FLOPs). The dotted shortcuts increase dimensions.

Image: He, Zhang, Ren & Sun (2015)

Learning outcomes

Hertie School

What we have covered

- Backpropagation (multi-class classification, several layers)
- Mini batch gradient descent
- Training process
- Vanishing gradient problem

Hertie School

Hertie School
Friedrichstraße 180
10117 Berlin, Germany
T +49 (0)30 259219-0
F +49 (0)30 259219-11
info@hertie-school.org
www.hertie-school.org